Домашнее задание
Неделя 4. Python для DE-задач

CSV, JSON, проверки качества и загрузка в PostgreSQL

Бизнес-кейс: «СеверМаркет»

Финансовая команда получает CSV с состоявшимися платежами от платёжного провайдера. Сейчас файлы исправляют вручную: удаляют ошибочные строки, меняют суммы и загружают данные повторно. Из-за этого невозможно объяснить пропуски и повторные записи.

Ваша задача - написать загрузчик, который сохраняет исходные файлы, проверяет каждую запись, отделяет отклонения и подтверждает результат в PostgreSQL. Отдельно подготовьте табличные данные из JSON-выгрузки заказов партнёра.

Выполните **7 заданий**. Задания 1, 2 и 4-7 составляют один загрузчик платежей. Задание 3 - отдельный обработчик заказов. Все компании и данные вымышлены.

Как начать

1. Распакуйте `DE_4_data.zip` в отдельную папку `week_04/` своего учебного репозитория. Запускайте команды из этой папки.

2. Используйте подготовленное окружение Python и учебную базу PostgreSQL. Установите зависимость из `requirements.txt`. Остальная обработка выполняется средствами стандартной библиотеки Python.

3. Перед заданием 4 выполните `sql/00_setup.sql` один раз. Он создаёт схему `de4_hw` и пустую таблицу. Если схема уже существует, используйте другую пустую учебную базу.

4. Исходные файлы в `data/raw/` не редактируйте. Скрипты сохраняйте в `src/`, свои проверочные SQL-запросы - в `sql/`.

Файлы в data/raw/	Назначение
`payments_day1.csv`	Первая выгрузка платежей
`payments_quality.csv`	Пограничные значения и ошибки отдельных записей
`payments_delta.csv`	Повторно доставленные и новые платежи
`payments_conflict.csv` / `payments_rollback.csv`	Конфликт с БД и проверка отката
`orders.json` / `orders_stream.jsonl`	Вложенные заказы в двух форматах
`fixtures/`	Ошибки структуры, кодировки и пустые файлы

Контрольные количества и суммы приведены на последней странице. Архив содержит все входные данные и DDL, но не готовые решения.

Договор с источником платежей

Одна запись CSV - один состоявшийся платёж. Кодировка **UTF-8 без BOM**, разделитель **точка с запятой**, символ кавычек ". Перенос строки внутри комментария допустим. Порядок колонок может отличаться.

Заголовок должен содержать ровно семь указанных ниже уникальных имён. Отсутствующее, дополнительное или повторяющееся имя, другой регистр и пробелы в имени колонки считаются ошибкой файла.

Исходное поле	Поле результата	Правило
`Payment ID`	`payment_id`	Непустая строка после strip(). Ведущие нули сохраняются
`Amount`	`amount`	Decimal. Целое или 1-2 знака после запятой. Значение > 0 и <= 9999999999.99
`Currency`	`currency`	Убрать крайние пробелы, привести к верхнему регистру. Допустимо только EUR
`Created At`	`created_at`	Момент создания платежа: ISO 8601 с явным смещением от UTC
`Paid At`	`paid_at`	Момент оплаты в том же формате. Не раньше created_at
`Status`	`status`	После strip() и casefold(): paid или оплачен. Оба значения преобразуются в paid
`Comment`	`comment`	Строка, в том числе пустая. Сохранить содержимое без изменений

Все значения, кроме комментария, обязательны. В сумме разрешены только цифры и необязательная десятичная запятая: без знака, пробелов внутри числа, разделителей тысяч, экспоненты и обозначения валюты. Точка в сумме этого CSV недопустима. Лишние десятичные знаки отклоняйте, округлять их нельзя.

При преобразовании сумм убирайте только крайние пробелы. Моменты времени переводите в UTC. Не подставляйте текущую дату, ноль или иной статус вместо ошибочного значения.

Повторы и допуск к загрузке

Если очищенный `Payment ID` встречается в файле несколько раз, отклоните **все записи с этим ключом**, включая совпадающие. Считайте ключи по всем записям с непустым идентификатором, даже если другие поля ошибочны.

Для платежей допускается **не более 2% отклонённых записей**. Ровно 2% разрешены. Пустой набор или превышение порога запрещает запись в БД. Причины отклонений сохраните до проверки порога.

Платёж является неизменяемым событием. Повтор уже известного БД ключа разрешён только при совпадении **всех семи подготовленных полей**. Изменение любого значения требует ошибки и отката текущей загрузки.

1. Прочитайте и преобразуйте CSV

Ситуация. Провайдер передаёт ведущие нули, статусы в разном регистре и комментарии с кавычками. Финансы требуют загрузить значения без ручных исправлений и потерь текста.

Что сделать

1. Напишите чтение через `csv.DictReader`. Используйте `pathlib`, контекстный менеджер `with`, заданную кодировку, `newline=""` и строгий режим CSV.

2. Проверьте заголовок до обработки данных. Лишние значения в записи или отсутствующее поле останавливают обработку всего файла. Пустое существующее поле проверяйте отдельно по правилам его значения.

3. Вынесите преобразование одной записи в функцию. Она получает исходный словарь и возвращает новый словарь с полями из договора. Функция не должна открывать файлы или соединение с БД.

4. Реализуйте правила сумм, статусов и времени. Сравнивайте created_at и paid_at только после успешного разбора обоих моментов и проверки часового пояса.

5. Чтение выполняйте по записям. Для поиска всех повторов в небольшом CSV разрешается сохранить исходные словари в список; обработка JSON Lines в задании 3 должна быть потоковой.

6. Соедините этот код с проверками задания 2. После полного чтения и классификации запишите принятые строки через `csv.DictWriter` в отдельный каталог текущего запуска.

Формат подготовленного файла

Путь: `data/processed/<run_id>/payments.csv`. Создавайте каталоги программно. Формат: UTF-8, разделитель `;`, заголовок в следующем порядке:

```
payment_id;amount;currency;created_at;paid_at;status;comment
```

Сумма записывается с точкой и двумя десятичными знаками. Временные значения - ISO 8601 со смещением `+00:00`. Комментарий должен пережить запись и повторное чтение без изменения кавычек, точки с запятой и переноса строки.

Что проверить и сохранить

Проверьте платежи `000001` и `000007`: ключи остаются строками с ведущими нулями, комментарии совпадают с исходными. В README приведите подготовленные значения для `000001` и объясните, почему число физических строк CSV не равно числу записей.

Существование подготовленного CSV само по себе не разрешает загрузку: окончательное решение принимает проверка порога из задания 2.

Справка: Python: csv.DictReader и csv.DictWriter

2. Отделите отклонения и проверьте качество

Ситуация. После подключения нового провайдера команда получает некорректные суммы и конфликтующие идентификаторы. Поддержке нужен файл с исходными значениями и понятной причиной по каждой отклонённой записи.

Что сделать

1. Реализуйте проверки обязательности, формата, диапазона, валюты, статуса, временных значений и их согласованности. Примените правило повторяющихся ключей до допуска записей к загрузке.

2. Каждая полностью прочитанная запись должна попасть ровно в одну группу: accepted или rejected. Для отклонённой записи достаточно первой причины, но обе записи с повторяющимся ключом должны получить причину DUPLICATE_KEY.

3. Сохраняйте отклонения как JSON Lines: одна запись об отклонении на одну физическую строку. Многострочный комментарий внутри JSON кодируется штатным модулем json.

4. После полного чтения проверьте баланс: read = accepted + rejected. Число rejected означает количество записей, а не количество обнаруженных ошибок.

5. Сохраните файл отклонений и итоговые счётчики, затем проверьте пустой набор и порог 2%. Ошибка сохранения отклонений тоже запрещает загрузку.

6. Обработайте `payments_day1.csv` и `payments_quality.csv`. Второй файл служит проверочным набором: даже корректная его часть не должна попасть в БД, если нарушен порог.

Обязательные сведения об отклонении

Поле	Содержание
`run_id`	Идентификатор текущего запуска
`source_file`	Имя или путь исходного файла
`record_number`	Номер записи данных с 1, без заголовка
`raw`	Исходный словарь до очистки и преобразования
`errors`	Непустой список: поле, код причины, понятное описание

Путь результата: `data/rejected/<run_id>/payments.jsonl`. Если отклонений нет, создайте пустой файл. Новый запуск должен иметь отдельный каталог, чтобы не затереть результат предыдущего.

Что показать в README

Для обоих файлов приведите read, accepted, rejected, долю отклонений и решение о допуске. Покажите причины двух записей с ключом `900019`. Объясните, почему некорректная сумма в одной из них не позволяет принять другую.

3. Подготовьте позиции заказов из JSON

Ситуация. Коммерческий отдел получает заказы партнёра как вложенные документы. Аналитику нужна таблица, в которой одна строка соответствует одной позиции заказа.

Договор с источником orders.json

Корень документа - объект с полями `export_date` и `orders`. `orders` должен быть массивом. Дата выгрузки описывает поставку и не является датой заказа. Идентификаторы заказов в корректной части предоставленных файлов не повторяются.

Поле	Тип и правило
`order_id`	Непустая строка; крайние пробелы удаляются
`customer`	Обязательный объект
`customer.customer_id`	Непустая строка, ведущие нули сохраняются
`items`	Обязательный массив; пустой массив разрешён
`items[].line_no`	Положительное целое число, уникальное внутри заказа
`items[].sku`	Непустая строка; крайние пробелы удаляются
`items[].quantity`	Положительное целое число
`items[].unit_price`	Строка с целым числом или 1-2 знаками после точки. Цена > 0 и <= 9999999999.99

Все цены указаны в EUR. Обязательные значения не могут быть `null`. Для unit_price удаляйте крайние пробелы, проверяйте формат и используйте Decimal. Здесь десятичный разделитель **точка**: правила платёжного CSV с запятой к этому источнику не относятся.

Что сделать

1. Прочитайте документ через `json.load`. Проверьте контейнеры, наличие обязательных полей и типы значений. Если нарушена структура или значение любого заказа, отклоните весь документ и не публикуйте подготовленный результат.

2. Вынесите обработку одного заказа в функцию. Разверните items в строки: `order_id, line_no, customer_id, sku, quantity, unit_price, line_amount`.

3. Рассчитайте `line_amount = quantity * unit_price` через Decimal. Заказ с пустым items считается корректным и даёт ноль строк позиций. Не создавайте для него искусственную позицию.

4. Запишите строки через DictWriter в `data/processed/<run_id>/order_items.csv`. Формат - UTF-8, разделитель `;`, цены и суммы с двумя знаками после точки.

Отдельно проверьте `fixtures/orders_invalid_structure.json`: первый корректный заказ не должен превратиться в частично успешный результат всего документа. Данные заказов не загружаются в таблицу платежей.

3. Тот же источник в JSON Lines

Партнёр также присылает поток `orders_stream.jsonl`. Каждая непустая физическая строка содержит самостоятельный заказ. Ошибка одного сообщения не должна мешать разобрать остальные сообщения с известными границами.

Правила обработки потока

1. Читайте файл в цикле по строкам, без `read()`, `readlines()` и загрузки всех заказов в список. Для непустой строки используйте `json.loads`, затем ту же функцию обработки заказа, что и для обычного JSON.

2. Пустую физическую строку пропускайте и учитывайте отдельным счётчиком. Она не считается принятым или отклонённым заказом.

3. Ошибка JSON или нарушение правил заказа отклоняет всю текущую строку. Не записывайте часть позиций заказа, если одна из его позиций некорректна. Для этого разрешено держать в памяти один заказ и его позиции.

4. Сохраните каждое отклонение в `data/rejected/<run_id>/orders.jsonl`: run_id, source_file, line_number, raw_text и причину. raw_text содержит исходную строку без завершающего переноса; при неверном JSON словаря raw ещё нет.

5. Позиции корректного заказа сразу записывайте в результирующий CSV. Не накапливайте весь результат перед записью. Для этого потока бизнес разрешает отдельные отклонения без порога 2%. Ошибки чтения или записи самого файла останавливают обработку.

6. После полного чтения создайте `summary.json` через json.dump: счётчики, количество позиций, сумма quantity и сумма line_amount. Денежную сумму сохраните как строку с двумя знаками после точки.

Различите четыре случая

Значение items	Действие
Ключ отсутствует	Отклонить заказ: нет обязательного поля
`null`	Отклонить заказ: неизвестно содержимое обязательного массива
`""`	Отклонить заказ: строка вместо массива
`[]`	Принять заказ, записать ноль позиций

Результат задания 3

Обычный JSON и корректные сообщения JSON Lines описывают одни и те же заказы. Сравните оба результата по ключу `(order_id, line_no)` и значениям всех полей. Сохраните результаты в отдельных каталогах запусков.

В README покажите счётчики orders_read, orders_accepted, orders_rejected, blank_lines, items_written, units и total_amount. Проверьте orders_read = orders_accepted + orders_rejected. Количество заказов и количество позиций сверяйте отдельно.

Справка: Python: json.load, json.loads, json.dump

4. Загрузите подготовленные платежи в PostgreSQL

Ситуация. Финансы готовы использовать файл только после подтверждения, что в БД оказались именно принятые платежи. Загрузка и её проверки должны завершаться одной транзакцией.

Что сделать

1. Подключитесь через **Psycopg 3**. Параметры PGHOST, PGPORT, PGDATABASE и PGUSER читайте из окружения. Пароль получайте безопасным способом из материала недели, например через getpass. Сам пароль не сохраняйте в коде и репозитории.

2. Используйте созданную таблицу `de4_hw.payment_events`. В транзакции создайте временный буфер по её структуре, включая ограничения и первичный ключ, с удалением при COMMIT.

3. Реализуйте два способа заполнения буфера: executemany с порциями по 25 записей и COPY через write_row. На принятых данных day1 проверьте оба способа в отдельных пробных транзакциях. В этих пробах постоянную таблицу не заполняйте.

4. Сверьте количество и сумму в буфере с принятыми записями Python. В окончательном загрузчике выберите один из проверенных способов заполнения буфера.

5. Перенесите данные в постоянную таблицу по правилу неизменяемых событий. Используйте уникальный payment_id и ON CONFLICT DO NOTHING. После вставки проверьте соответствие всех принятых записей содержимому целевой таблицы.

6. До COMMIT сверьте ключи, все значения полей, количество и сумму по ключам текущей загрузки. При несовпадении выбросите исключение. Успех фиксируйте только после завершения транзакции.

7. Выполните первую постоянную загрузку `payments_day1.csv`. Из другого подключения подтвердите результат SQL-запросом. Сохраните свои запросы в `sql/checks.sql`.

Условия приёмки

Значения для execute и executemany передавайте параметрами. Не подставляйте комментарий или другие входные значения в строку SQL. При COPY передавайте подготовленные Python-значения через write_row.

Размер порции не задаёт границу транзакции. Промежуточных COMMIT между порциями быть не должно. Соединение и курсор должны закрываться и после ошибки.

Сверка проводится по принятым ключам текущего файла. Общая сумма всей таблицы станет больше суммы текущей поставки после нескольких успешных загрузок.

Результат задания

Работающая загрузка, две функции заполнения буфера и SQL-сверки. В README покажите результаты обеих проб буфера и первой загрузки. Подтвердите сохранность комментария с апострофом и многострочного комментария.

Справка: Psycopg: COPY

Справка: Psycopg: управление транзакциями

5. Обработайте повторную доставку и конфликт

Ситуация. Провайдер повторно отправил вчерашний файл, затем дослал новые платежи вместе с уже известными. Ещё в одной поставке он изменил время оплаты существующего события. Финансы запрещают незаметно перезаписывать историю.

Продолжайте работу с таблицей после задания 4. Между следующими запусками таблицу не очищайте.

Последовательность проверки

1. Повторно запустите `payments_day1.csv`. Отклонения снова должны сохраниться в новом каталоге запуска. Принятые платежи уже существуют, поэтому новых вставок не будет.

2. Загрузите `payments_delta.csv`. В нём два совпадающих платежа из первой поставки и три новых. Смещение времени у повторных записей отличается по записи, но обозначает тот же момент. Сравнивайте подготовленные значения в UTC.

3. Перед загрузкой `payments_conflict.csv` зафиксируйте состояние таблицы и данные платежа `000002`. В файле изменён его Paid At, а сумма осталась прежней. Вместе с ним пришёл новый платёж `000901`.

4. Запустите конфликтный файл. Загрузчик должен обнаружить расхождение и откатить всю текущую транзакцию. Платёж `000002` остаётся прежним, а `000901` не появляется.

5. Проверьте последствия из отдельного подключения. Сохраните в README сравнение состояния до и после каждого запуска.

Какие показатели различать

Показатель	Смысл
`accepted`	Записи, прошедшие проверки текущего файла
`inserted`	Новые строки, зафиксированные текущим запуском
`matched`	Записи БД, соответствующие принятым ключам и значениям
`accepted_amount`	Сумма принятых записей в Python
`matched_amount`	Сумма записей БД по принятым ключам

Успешный повтор может иметь accepted > 0 и inserted = 0. При неуспешном запуске вставки, отменённые откатом, не считаются зафиксированным результатом.

Объясните в README

Почему ON CONFLICT DO NOTHING само по себе недостаточно? Почему проверка только количества и суммы пропустила бы изменение времени в конфликтном файле? Чем повтор ключа внутри одного источника отличается от повторной доставки уже загруженного события?

6. Проверьте отказы и откат

Ситуация. Операционная команда получает сообщение «загрузка завершена», хотя файл не прочитан или сохранена только часть поставки. Перед передачей загрузчика нужно воспроизвести такие ситуации и проверить фактическое состояние БД.

Ошибки входного файла

Для каждого случая ниже ожидается FAILED и отсутствие новых изменений в таблице. Уже сохранённые платежи должны остаться прежними. Файлы находятся в `data/raw/fixtures/`.

Сценарий	Что проверить
`empty.csv`	Нет заголовка, обработка не начинается
`header_only.csv`	Заголовок верен, но записей нет; пустая загрузка запрещена
`missing_column.csv`	Не хватает обязательной колонки
`duplicate_column.csv`	Повторяющееся имя колонки
`short_row.csv` / `extra_field.csv`	Число полей записи не соответствует заголовку
`broken_quotes.csv`	Повреждены кавычки; нельзя надёжно продолжить CSV
`invalid_utf8.csv`	Ошибка декодирования файла
Несуществующий путь	Ошибка открытия входного файла
`payments_quality.csv` из data/raw/	Превышение порога отклонений после сохранения причин

Контрольный сбой внутри транзакции

1. Используйте `payments_rollback.csv`. В учебной копии загрузчика временно вызовите RuntimeError **после вставки новых строк в постоянную таблицу, но до COMMIT**. Исключение должно выйти из контекстного менеджера соединения.

2. Из другого подключения подтвердите: все три новых ключа отсутствуют, прежние записи не изменились. Удалите контрольный сбой и повторите файл: теперь все три платежа должны сохраниться.

Ошибка подключения

Укажите заведомо неверное имя учебной базы и запустите тот же файл. Скрипт должен завершиться ошибкой, закрыть ресурсы и не сообщать об успешной загрузке. Верните правильную настройку после проверки.

В README приведите таблицу «сценарий / ожидаемый результат / фактический результат / состояние БД». При неполном чтении не выдавайте частичные счётчики за итог всего файла.

7. Соберите запуск и подготовьте передачу

Ситуация. Коллега должен запустить обработку другого файла без редактирования Python-кода и понять результат по журналу и README.

Сборка скрипта

1. Разделите чтение, проверку, преобразование, сохранение отклонений и загрузку на функции. Операции с файлами и БД запускайте из main(), а не при импорте модуля.

2. Добавьте обязательный аргумент `--input` через argparse и точку входа `if __name__ == "__main__":`. Команда --help должна работать без подключения к БД.

3. Для платежей используйте `src/load_payments.py`, для задания 3 - `src/prepare_orders.py`. Второй скрипт выбирает обычный JSON или JSON Lines по расширению входного файла.

4. Создавайте новый run_id для каждого запуска. Сохраняйте summary.json в каталог результата; Decimal сериализуйте как строку, datetime - как ISO 8601. Результаты отдельных запусков не смешивайте.

```
python src/load_payments.py --help
python src/load_payments.py --input data/raw/payments_day1.csv
python src/prepare_orders.py --input data/raw/orders.json
python src/prepare_orders.py --input data/raw/orders_stream.jsonl
```

Журнал и статусы

Используйте logging. Записывайте run_id, этап, имя файла, счётчики, суммы и итоговый статус. При ошибке фиксируйте тип и понятную причину. Не выводите пароль, строку подключения с секретом, исходные записи или текст комментариев в общий журнал.

Статус	Когда использовать
`SUCCESS`	Обязательные этапы завершены, отклонений нет
`SUCCESS_WITH_REJECTS`	Обязательные этапы завершены, допустимые отклонения сохранены
`FAILED`	Не прочитан файл, превышен порог, не сохранены отклонения, произошёл конфликт или иной сбой

Для первых двух статусов код завершения 0, для FAILED - 1. В загрузчике платежей успешный статус возможен только после COMMIT. В обработчике заказов - после полной записи результата. Общий обработчик исключений находится на внешнем уровне и завершает неуспешный запуск.

Что сдать

В папке week_04/ сохраните код, зависимости, SQL-проверки и README. В README укажите подготовку БД, названия переменных окружения, точные команды, правила источников и результаты заданий 1-6. Приложите подготовленные CSV, файлы отклонений и summary.json контрольных запусков. Для неуспешных запусков приложите безопасный фрагмент журнала.

Сдайте ссылку на эту папку репозитория. Секреты, виртуальное окружение и локальные служебные файлы в репозиторий не добавляйте.

Контрольные значения для самопроверки

Значения ниже относятся к предоставленным файлам. Суммы платежей и стоимости позиций указаны в EUR. Сумма принятых строк не означает, что загрузка разрешена или уже зафиксирована.

Платежи после проверки и преобразования

Файл	Прочитано	Принято	Отклонено	Сумма принятых
payments_day1.csv	100	98	2	5077.50
payments_quality.csv	20	4	16	10000000135.50
payments_delta.csv	5	5	0	388.26
payments_conflict.csv	2	2	0	113.33
payments_rollback.csv	3	3	0	60.00

У quality доля отклонений 80%, поэтому принятые строки не загружаются. У conflict проверки входного файла пройдены, но сравнение с ранее загруженными данными должно остановить транзакцию.

Состояние постоянной таблицы по порядку запусков

После запуска	Новых зафиксировано	Строк в таблице	Общая сумма
Первый day1	98	98	5077.50
Повтор day1	0	98	5077.50
delta	3	101	5333.01
conflict, ожидаемый отказ	0	101	5333.01
rollback с контрольной ошибкой	0	101	5333.01
rollback после удаления ошибки	3	104	5393.01

Начальное состояние перед первым day1 - пустая таблица. Пробы заполнения временного буфера, проверки файлов fixtures и quality не меняют постоянную таблицу.

Заказы из JSON и JSON Lines

Показатель	orders.json	orders_stream.jsonl
Непустых записей заказов	5	9
Принятых / отклонённых заказов	5 / 0	5 / 4
Пустых физических строк	Не применяется	1
Подготовленных позиций	6	6
Количество единиц товара	13	13
Сумма line_amount	140.10	140.10

Для JSON Lines отдельно проверьте 10 физических строк: 1 пустая и 9 непустых. Заказ с пустым массивом включается в принятые заказы, но не увеличивает количество позиций.